First Project Report

1. Introduction

The goal of this project is to optimize the staff needed for an airport security checkpoint. The airport wants to minimize operational costs by having fewer service stations, while maintaining a high quality of service for the customers (travelers). Specifically, on this project, we aim to determine the optimal number of service stations (k) needed for the average traveler's waiting time to be below one minute.

2. Model Description

2.1 Basic Model:

To simulate the airport security checkpoint with an M/G/k system model, meaning a set of k independent M/G/1 queues. We implemented an Object-Oriented discrete-event simulation in Python. The model is structured based on three classes: *Traveler*, *Queues*, and *Airport*, which interact to represent the queuing system.

2.1.1 Assumptions:

- The system uses the First-in First-out (FIFO) method. Travelers within each queue are served based on the order in which they arrived.
- Travelers who arrive instantly know the length of each queue and are assigned or choose the shortest queue automatically.
- There is no limit on the number of travelers who can be waiting in a queue
- Travelers can not move between queues, and they also can not leave the queue.

2.1.2 Parameters

- **Arrival rate (λ):** 10 per min following an exponential distribution

- **Service mean (μ_1)**: 0.5 min following a truncated normal distribution
- **Service Standard Deviation (σ):** 10 seconds (need to convert to mins probably)
- **Number of stations (k)**

2.2 Extended Model:

To reflect real-world constraints, we introduced a Senior Security Officer. There is a 3% probability that a traveler requires secondary screening. This creates a blocking condition: the traveler occupies their primary service station (preventing others from being served) until the single, shared Senior Officer completes the additional inspection (Mean = 2.0 mins).

3. Implementation Details

To measure the average traveler waiting time, we implemented the class Traveler to store each passenger's arrival_time, allowing us to calculate each passenger's waiting time as the difference between the service start time and arrival time.

Additionally, the *Queues* class represents and controls each singular queue and service station. In which we implemented the *update_info* method to measure the average queue length using a time-weighted average (accumulating the area under the queue-length curve) to ensure the reported average reflects the true state of the system over time.

Finally, the *Airport* class manages k service stations. It also assigns passengers to the shortest queue. When the exponential arrival distribution generates a new event, the *Airport* scans all *Queues* instances and assigns the *Traveler* to the queue with the minimum length (by calculating *min(len(queue)*)

4. Queuing Theory:

Our simulation uses a heap priority queue (using Python's heapq module) to manage the

time. The arrivals are modeled using the exponential distribution, reflecting the memoryless property of independent arrivals assumed in standard queuing theory. And service is modeled using a truncated normal distribution that ensures positive service times, while maintaining the variance in passengers' screening times/speeds.

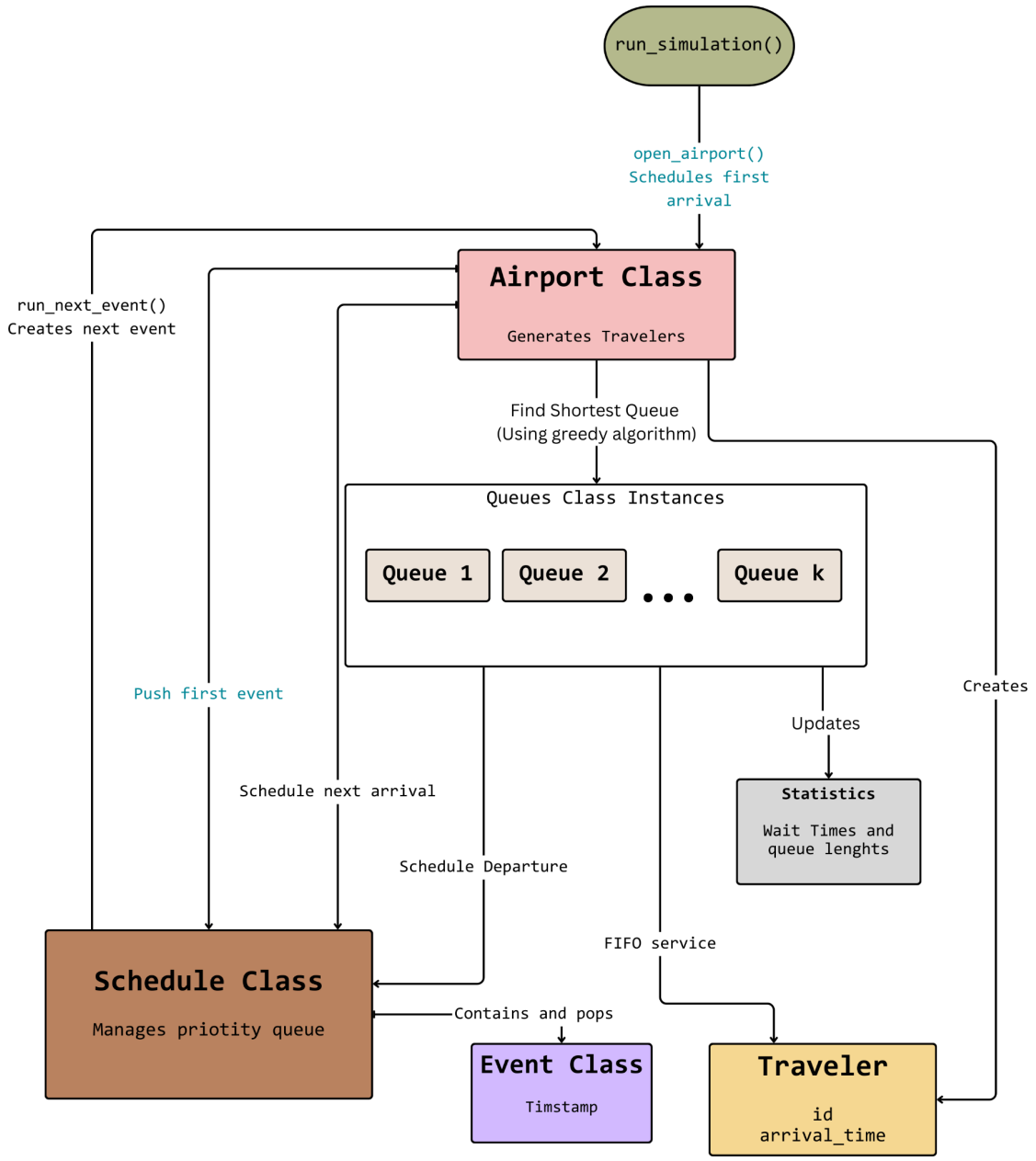

Figure 1: Diagram that illustrates the simulation Class architecture and logic flow. The Schedule drives the event into the loop. The Airport generates the travelers and assigns them to the Queues.

5. Verification and Validation:

To demonstrate that our simulation is correctly implemented, we performed theoretical verification, result comparison with given parameters, and boundary conditions testing.

5.1 Distribution verification:

We confirmed that the random number generators correctly produced the specified distributions

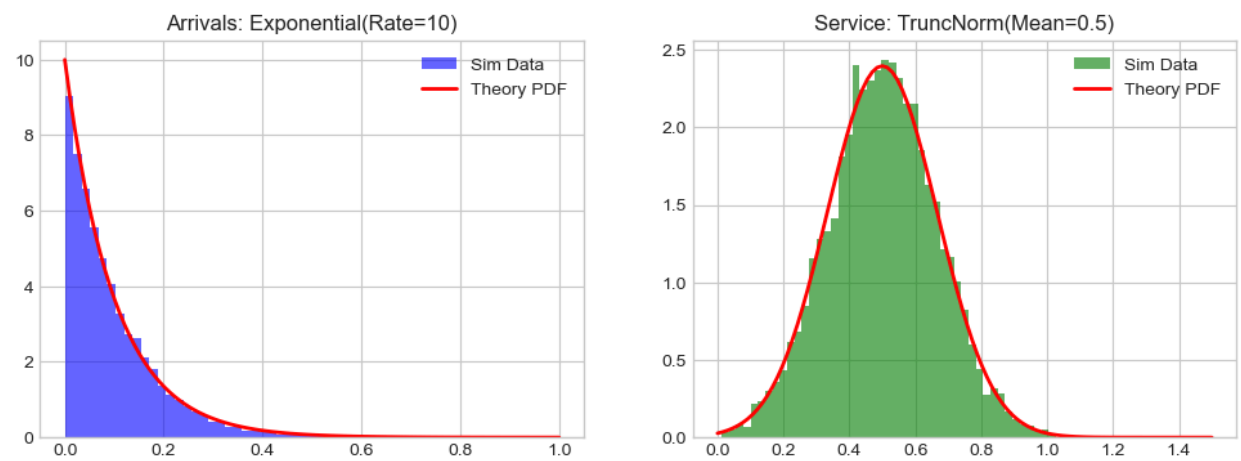

Figure 2: Plot showing that the random number generators were correctly producing the distributions, Exponential for arrivals (left), Truncated Normal for service (right), compared to the theoretical

5.2 Given Parameters

```
Test 1: Uniform Service with 6 queues
Running
100%|          | 1000/1000 [02:54<00:00,  5.72it/s]
 Result: 0.49850000000000005
Passed
Test 2: Truncated Normal with 14 Queues
Running
100%|          | 1000/1000 [16:40<00:00,  1.00s/it] Result: 0.5810714285714286
Passed
```

Figure 3: Results from parameter testing with the assigned parameters and expected results confirm that our simulation is within the passing boundaries.

To eliminate the error introduced by starting with an empty airport, we used independent

replications: running 1000 separate trials and taking a snapshot of the system state at t=100 minutes.

5.3 M/G/1 Theoretical Verification

We validated the queueing logic by comparing the basic model against theoretical OK formula results. We configured the simulation as a single queue (k=1) with a scaled arrival rate (lambda/k). As shown in Appendix B, the empirical results matched the theoretical predictions within statistically acceptable margins.

5.4 Robustness Check

We tested the extended model under boundary conditions. When the probability of secondary screening was set to 0, the model correctly reproduced the Basic Model results. Additionally, when the arrival rate exceeded the service rate (rho > 1), the queue length grew linearly towards infinity, confirming the model correctly handles system overload.

6. Main question: How many service stations should airport security have?

6. 1 Theoretical Analyses:

Using queuing theory, we can theoretically approximate the airport security system as a collection of *k* parallel M/G/1 queues. We use M/G/1 instead of M/G/k because each queue is approximately independent and has only one server (one traveler is served at a time), and routing is state dependent (each traveler joins the shortest queue). Each queue receives arrivals approximately split evenly ("join shortest queue") at a rate of $\frac{\lambda}{k}$.

Under these approximations, queueing theory predicts that the mean waiting time and mean queue length depend strongly on the traffic intensity $\rho = (\frac{\lambda}{k})E[S]$, diverging as ρ

approaches 1. This is consistent with what we've seen in class, if the traffic intensity increases, a bottleneck phenomena is happening and the queue lengths, and respectively the waiting time, increase sharply.

There are several components of the system that can be approximated analytically using queueing theory. In particular, we can estimate the expected waiting time W_q per queue, the expected average queue length L_q per queue, and the minimum staffing level required for stability, given by the condition $\rho < 1$. These quantities follow directly from classical M/G/1 results and provide useful predictions for system performance.

However, the full airport security model (including extended model) cannot be solved exactly using theory (only approximated). Limitations include: join-shortest-queue routing makes the queues dynamically dependent, violating the independence assumptions required for analytical results. In addition, the maximum queue length during a day is an extreme-value metric that is not tractable analytically in this context. Finally, the extended model introduces a shared secondary screening resource, which couples all queues through a common bottleneck and further breaks the assumptions of standard queueing models. As a result, we need to implement a simulation to evaluate these aspects of system performance more accurately.

6.1.1 M/G/1 waiting time formula (from class):

In our code we are calculating waiting time before service as *wait_time = schedule.now - traveler.arrival_time* which represents the queueing delay, denoted as W_q, not total time in the system.

For an M/G/1 queue, the Pollaczek–Khinchine formula gives:

$$W_q = \frac{\lambda_q E[S^2]}{2(1-\rho)} \text{ where } \rho = \lambda_q E[S]$$

And mean queue length (waiting line) is:

$$L_q = \lambda_q W_q$$

These are the "theory" versions of the two main metrics our simulation estimates:

- mean waiting time
- mean queue length (average over time)

In the following section, we focus on the stability condition derived from M/G/1 queueing theory to address the staffing problem directly. Specifically, we use the condition $\rho < 1$ to derive the minimum number of service stations required for stability, first for the basic airport model and then for the extended model with additional screening, under both parameter sets provided.

6.1.2 Minimum number of service stations (k) for the base model:

First we will compute the expected (average) service time denoted as $E[S]$, and expected square of the service time $E[S^2]$

Parameter Set 1: $S \sim Uniform(0, 1)$

This distribution means that all service times between 0 and 1 minute are equally likely.

For a $Uniform(a, b)$ random variable:

$$E[S] = \frac{a+b}{2}, E[S^2] = \frac{a^2 + ab + b^2}{3}$$

Here a=0, b=1:

$$E[S] = 0.5, E[S^2] = \frac{1}{3}$$

Traffic intensity per queue:

$$\rho = \lambda_q E[S] = \frac{10}{k} \cdot 0.5 = \frac{5}{k}$$

Stability condition $\rho < 1$ gives:

$$\frac{5}{k} < 1 \Rightarrow k > 5 \Rightarrow k \geq 6$$

Stability condition: $k \geq 6$

So, we need equal to or more than 6 queues to reach "minimum staffing for stability" for Parameter set 1.

Parameter Set 2: $S \sim TruncNorm\,(-1, \infty;\ \mu = 1, \sigma = 1)$

This is a normal(1,1) truncated below at 0 (because -1 standard deviations below mean corresponds to 0 in real time units, which makes no sense for modeling real world systems).

So $S \geq 0$. This makes the mean larger than 1.

Using truncated normal moment formulas: $E[S] \approx 1.288$

Then: $\rho = \frac{10}{k} \cdot 1.288 = \frac{12.88}{k}$

Stability condition: $\frac{12.88}{k} < 1 \Rightarrow k > 12.88 \Rightarrow k \geq 13$

So, Parameter set 2 needs about 13+ service stations in order to be stable.

6.1.3 Minimum number of service stations (k) for the extended model:

In the extended model, a traveler's "primary service completion" is not always the true completion of the station's work. With probability p_s, the queue must wait for the senior officer (which is shared among all stations) and the station remains occupied during this time. From a queueing-theory perspective, this means the effective service time at each station is no longer just S (primary screening time), but $S_{eef} = S + I \cdot B$, where $I \sim Bernoulli(p_s)$ indicates whether the traveler is flagged for extra screening, and B is the blocking time caused by the secondary process (waiting for the officer + officer screening time). This matches our implementation: the station is not freed until the officer finishes. In other words, we could analyse the shared officer as an M/G/1 bottleneck.

The rate at which flagged travelers arrive at the senior officer is: $\lambda_2 = \lambda p_s$

Let's say extra-screening time is D (the truncated normal with parameters μ_2,σ_2, truncated

at 0). Then, the officer utilization is $\rho_2 = \lambda_2 E[D]$

In order for the extended model to be stable, we need both the officers' queue to be stable (otherwise it grows without bound and all stations increasingly block) and each station queue to be stable (derived above).

If $\rho_2 < 1$, we can approximate the officer's queueing delay using the M/G/1 waiting-time formula: $W_{q,2} = \frac{\lambda_2 E[D^2]}{2(1-\rho_2)}$

So a flagged traveler causes an expected blocking time roughly equal to: $E[B] \approx W_{q,2} + E[D]$.

So the catch here is that the officer's own congestion directly increases station blocking, even if the primary stations are many.

Because a traveler "triggers" blocking only with a p_s probability, then we can convert officer delay into an effective station service time:

$$E[S_{eff}] = E[S] + p_s E[B] \approx E[S] + p_s(W_{q,2} + E[D]).$$

This gives us a theoretical way to translate the extended model into a stability condition similar to the one of the base model. Now, we can apply the station stability condition to solve for k (number of stations).

$$\rho_{eff} = \lambda_q E[S_{eff}] = \frac{\lambda}{k} E[S_{eff}].$$

So for stability we need:

$$\rho_{eff} < 1 \Rightarrow k > \lambda E[S_{eff}]$$

So this means that the extended model "raises" the required staffing level because it increases the average "busy time" per traveler through secondary-screening blocking.

The extended model parameters are: $p_s = 0.03$ and $D \sim truncated\ normal$ with $\mu_2 = 2, \sigma_2 = 2$ (truncated at 0). With $\lambda = 10$,

$$\lambda_2 = 10 \cdot 0.03 = 0.3 per\ minute$$

From truncated-normal moments (lower bound cutoff at 0), we can approximate:

$$E[D] \approx 2.575, \quad E[D^2] \approx 9.155.$$

Officer utilization: $\rho_2 = 0.3 \cdot 2.575 \approx 0.773 < 1$ => is stable but a bit loaded (value close to 1)

Officer queueing delay: $Wq, 2 \approx \frac{0.3 \cdot 9.155}{2(1-0.773)} \approx 6.0 min.$

Expected blocking time per flagged traveler: $E[B] \approx W_{q,2} + E[D] \approx 6.0 + 2.6 = 8.6 min$

Thus the effective service time increase per traveler is $p_s E[B] \approx 0.03 \cdot 8.6 \approx 0.258 min$

So, overall: $E[S_{eff}] \approx E[S] + 0.258$

Now we can calculate the stability threshold using the two sets of parameters:

Parameter set 1 (primary service $S \sim U(0,1)$)

$$E[S] = 0.5 \Rightarrow E[S_{eff}] \approx 0.5 + 0.258 = 0.758.$$

Stability: $k > \lambda E[S_{eff}] = 10(0.758) = 7.58 \Rightarrow k \geq 8$

Interpretation: compared to the base-model threshold ($k \geq 6$), adding secondary screening pushes the minimum stable staffing up because the stations are blocked by the officer process.

Parameter set 2 (primary service truncated normal with $E[S] \approx 1.288$)

$$E[S] \approx 1.288 \Rightarrow E[S_{eef}] \approx 1.288 + 0.258 = 1.546.$$

Stability: $k > 10(1.546) = 15.46 \Rightarrow k \geq 16$

This results aligns with the empirical simulation outcome: even at k=14, waiting times remain pretty large, which is consistent with the theory suggesting that secondary screening makes the system behave “as if” each server is slower on average, because of the bottleneck effect on individual “basic” security lines.

6. 2 Theoretical Analyses Conclusion

Using M/G/1 queueing theory, we derived stability conditions that provide lower bounds on the number of service stations required for the airport security system to function reliably. For the **base model**, theoretical analysis predicts minimum staffing levels:

- Parameter Set 1: $k \geq 6$

- Parameter Set 2: $k \geq 13$

Extending the model to include secondary screening introduces a shared bottleneck that increases the effective service time at each station, raising the theoretical stability thresholds to:

- Parameter Set 1: $k \geq 8$
- Parameter Set 2: $k \geq 16$

Combining theoretical bounds with simulation-based performance metrics (mean waiting time, average queue length, and maximum queue length), we conclude that staffing ~ 10 service stations provides a robust and practical solution: it meets the service-quality target, mitigates congestion risk, and offers a safety buffer against stochastic fluctuations introduced by secondary screening.

7. Empirical Analysis:

To answer the main question of the assignment, we ran the extended model on Appendix A over a range of service stations from k =5 to k=17, and we performed 50 independent trials to calculate 95% Confidence Intervals.

7.1 Stability Analysis:

A critical finding was determining the tipping point of stability.

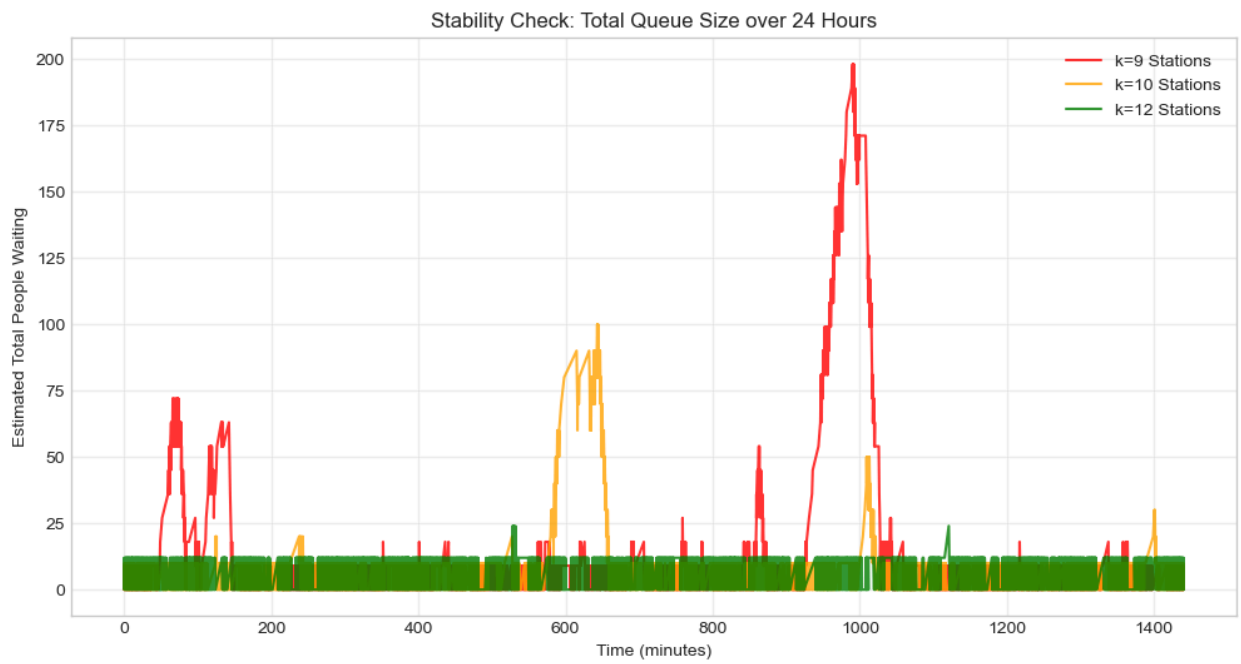

Figure 4: Total queue size over a 24-hour period. At k=9 (Red), the queue shows an upward trend, indicating potential instability. At k =10 (Orange), the system stabilizes

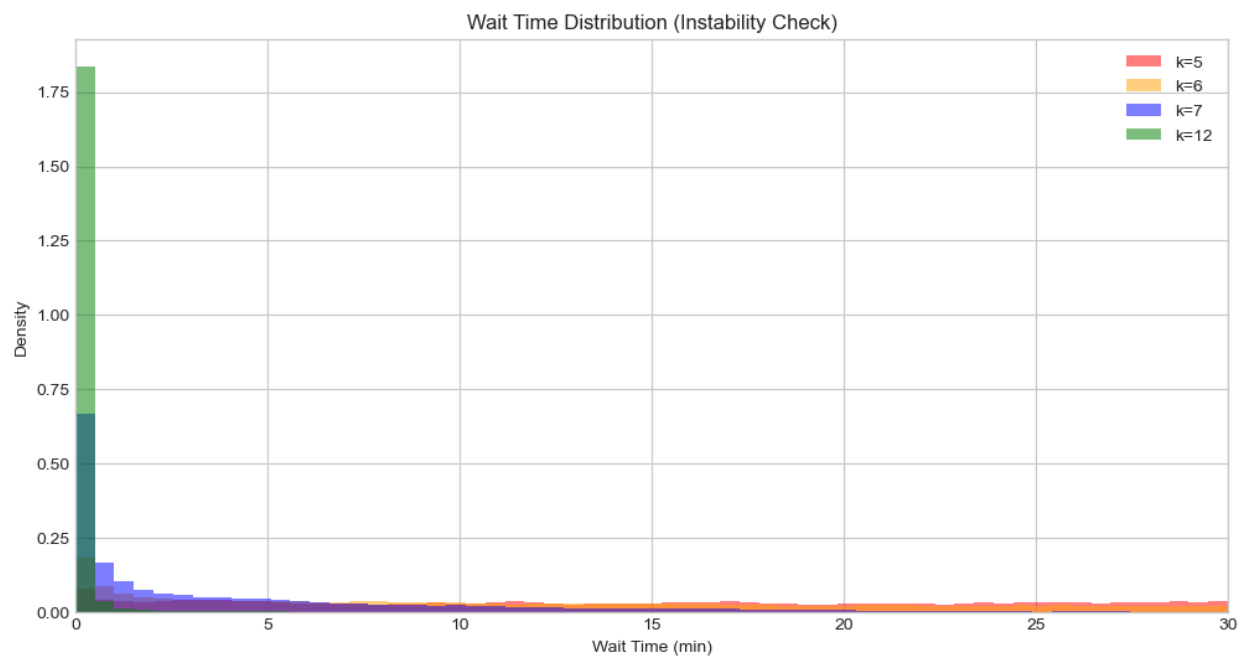

Figure 5: Distribution of wait times. At low staffing (k<8), wait times are widely distributed with a long tail of extreme delays. At k=12, wait times cluster tightly near zero.

7.2 Optimization Metrics

7.2.1 Metric 1: Average traveler waiting time given the number of service stations.

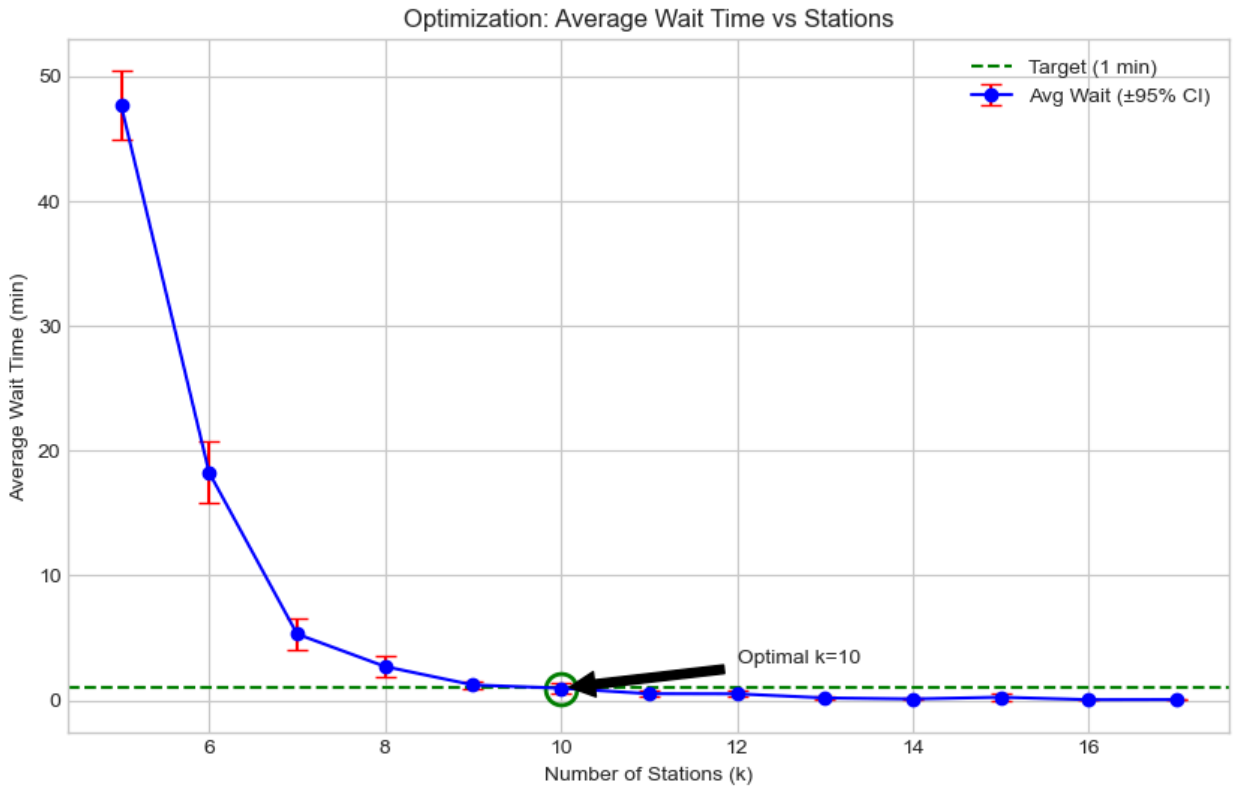

Figure 6: Average wait time vs. staffing level. The red bars represent the 95% Confidence Interval. The green line marks the 1-minute target.

As shown in Figure 6, the waiting time drops exponentially as stations are added. At k=9 average wait is ~1.5 mins, violating the target. At k = 10, the Average wait drops to 0.80 minutes (±0.15), meeting our target.

7.2.2 Metric 2 and 3: Average queue length during a day, given the number of service stations and maximum queue length during a day, given the number of service stations.

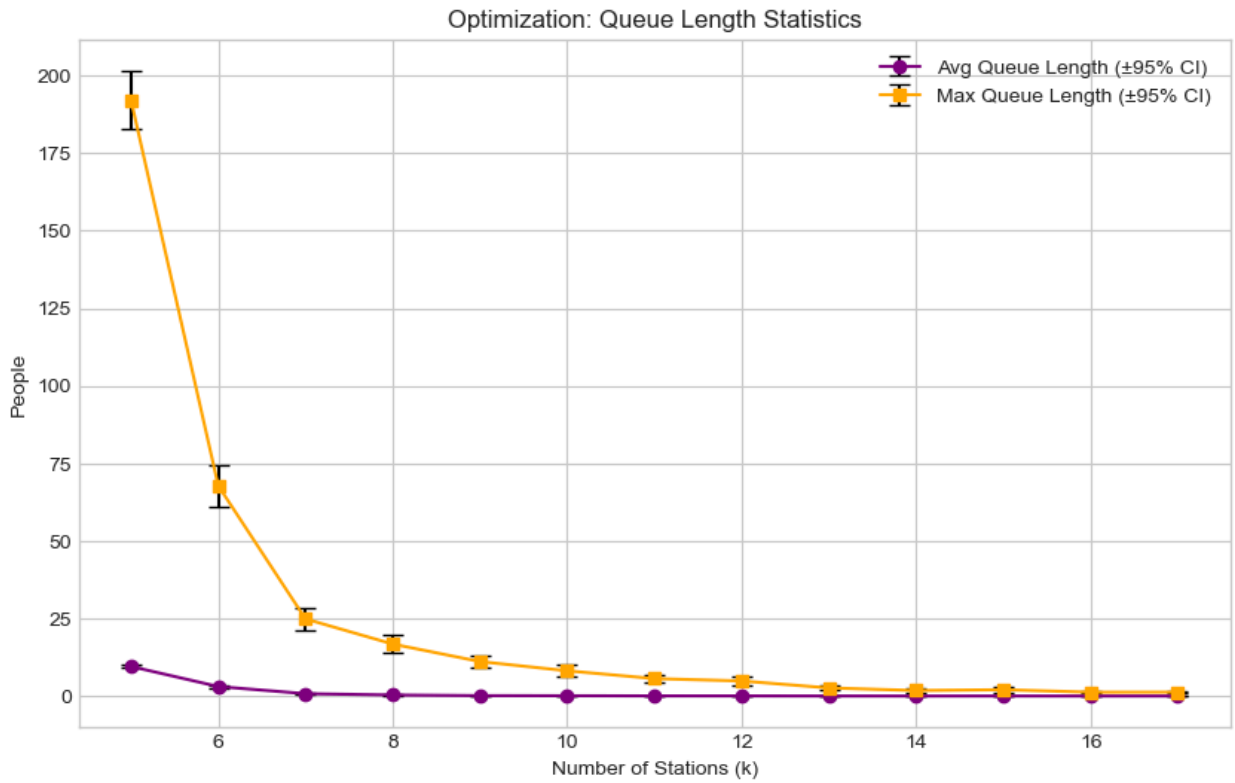

Figure 7: Average Queue Length (Purple) and Maximum Queue Length (Orange).

Figure 7 highlights the physical constraints of the airport. While the average queue length decreases smoothly, the Maximum Queue Length (Orange Line) reveals a critical insight. At k=9, the maximum queue often spikes to over 60 people, causing severe overcrowding. At k=10, this maximum drops significantly, indicating a much more manageable physical flow.

4.3 Theoretical Expansion (Utilization Analysis):

To satisfy the requirement of analyzing the system analytically, we compared the Extended Model's performance against the theoretical M/G/1 prediction across different utilization levels (rho).

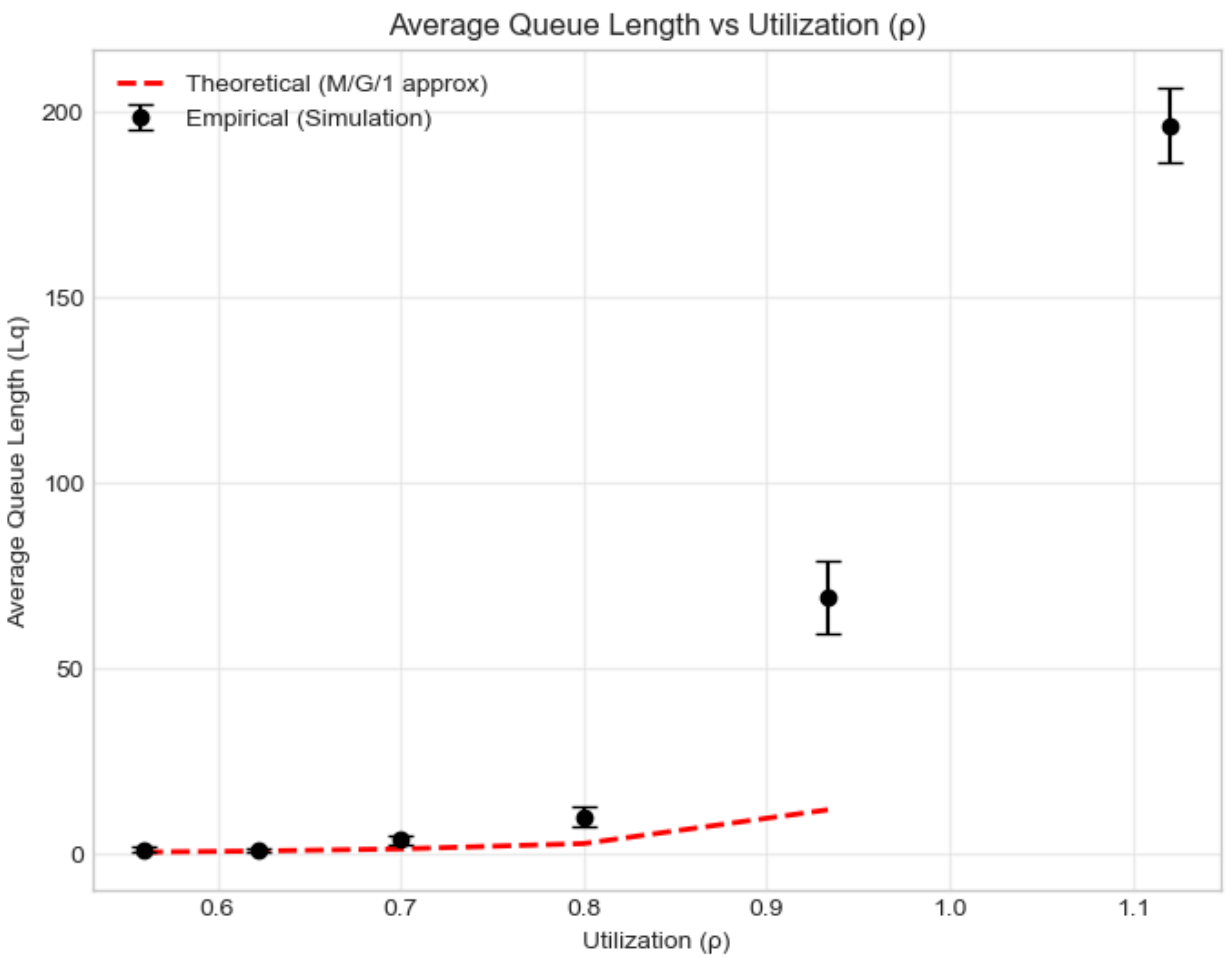

Figure 6: Comparison of Theoretical M/G/1 predictions (Red Line) vs. Empirical Simulation results (Black Dots).

As shown in Figure 6, our simulation follows the theoretical curve at low utilization ($p<0.7$). However, at high utilization, the empirical queue length grows faster than the standard M/G/1 formula predicts. This occurs because the standard formula does not account for the blocking effect of the senior officer when he is busy, because multiple servers can become blocked simultaneously, thus reducing the system's capacity in a way that a simple independent queue scenario can not capture. This confirms the necessity of a simulation for this complex scenario.

Discussion and recommendation

Interpretation:

This results in three zones:

- **Unstable zone (k < 10):** The system is overwhelmed, queues grow uncontrollably (Figure)
- **Optimal zone (k =10):** As concluded by the code and as it can be observed in the plots, the system stabilizes. The average wait time meets our target, and the maximum queue length is contained within safe limits.
- **The too much staff zone (K >12):** Wait times are near zero, and the addition of a staff member incurs costs without improving the passenger experience significantly (wait time)

Recommendation:

Our report concludes that we recommend staffing 10 service stations. While k=9 is mathematically close to the target, the stability analysis proves it is too risky; a small spike in secondary screenings would cause the queues to explode. 10 service stations provide a safe buffer to handle stochastic shocks to the system, keeping wait times under 1 minute and ensuring physical stability in the terminal.

Appendix:

Some of the code referenced in the assignment. The complete code can be accessed in the attached jupyter notebook.

Appendix A: The following code implements the Basic and Extended Model, including the Schedule, Airport, and Queues classes.

```
#importing libraries and use of main classes adapted from CS166 class
content
import heapq
import scipy.stats as sts
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

#for the plot styles
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)

class Event:
    '''
    Store the properties of one event in the Schedule class defined
below. Each
    event has a time at which it needs to run, a function to call
when running
    the event, along with the arguments and keyword arguments to pass
to that
    function.
    '''
    def __init__(self, timestamp, function, *args, **kwargs):
        self.timestamp = timestamp
        self.function = function
        self.args = args
        self.kwargs = kwargs

    def __lt__(self, other):
        '''
```

```
        This overloads the less-than operator in Python. We need it
so the
        priority queue knows how to compare two events. We want
events with
        earlier (smaller) times to go first.
        '''
        return self.timestamp < other.timestamp

    def run(self, schedule):
        '''
        Run an event by calling the function with its arguments and
keyword
        arguments. The first argument to any event function is always
the
        schedule in which events are being tracked. The schedule
object can be
        used to add new events to the priority queue.
        '''
        self.function(schedule, *self.args, **self.kwargs)


class Schedule:
    '''
    Implement an event schedule using a priority queue. You can add
events and
    run the next event.

    The 'now' attribute contains the time at which the last event was
run.
    '''

    def __init__(self):
        self.now = 0  # Keep track of the current simulation time
        self.priority_queue = []  # The priority queue of events to
run

    def add_event_at(self, timestamp, function, *args, **kwargs):
        # Add an event to the schedule at a particular point in time.
```

```
        heapq.heappush(
            self.priority_queue,
            Event(timestamp, function, *args, **kwargs))

    def add_event_after(self, interval, function, *args, **kwargs):
        # Add an event to the schedule after a specified time
interval.
        self.add_event_at(self.now + interval, function, *args,
**kwargs)

    def next_event_time(self):
        return self.priority_queue[0].timestamp

    def run_next_event(self):
        # Get the next event from the priority queue and run it.
        event = heapq.heappop(self.priority_queue)
        self.now = event.timestamp
        event.run(self)

    def __repr__(self):
        return (
            f'Schedule() at time {self.now}min ' +
            f'with {len(self.priority_queue)} events in the queue')

    def print_events(self):
        print(repr(self))
        for event in sorted(self.priority_queue):
            print(f'  ⏱ {event.timestamp}min:
{event.function.__name__}')
```

```
import scipy.stats as sts

class Traveler:
  '''
  to track individual arrival times because assignment asks to
measure "Average traveler waiting time"
  '''
```

```
  def __init__(self, arrival_time, traveler_id):
        self.arrival_time = arrival_time
        self.id = traveler_id
        self.service_time = 0
        self.departure_time = 0

class Queues:
  """
  stores and manages each one of the lines individually by making
them an M/G/1 (one line and one server)

  """
  def __init__(self, queue_id, service_distribution):
        self.queue_id = queue_id
        # Store the service-time distribution for an M/G/1 queue
        self.service_distribution = service_distribution

        #list for travelers waiting people in the queue
        self.queue = []
        self.people_being_served = 0 #maybe change it to a list??]

        #statistics
        #list of waiting times from travelers
        self.waiting_times = []
        self.queue_lenghts = []

        #for average time
        self.last_update_time = 0
        self.area_under_queue_curve = 0

  #create a less than method to compare lenghts of queues
slef.len(people inqueue +people_incounter ) <other(people in queue
people in the qcounter)

  def update_info(self, current_time):
    '''
    Updates the time-dependent information for the queue.
    '''
```

```
    #area under the curve from last update
    time_delta = current_time - self.last_update_time
    current_length = len(self.queue)
    #+ self.people_being_served
    self.area_under_queue_curve += current_length * time_delta
    self.last_update_time = current_time

    #log the data point
    self.queue_lenghts.append((current_time, current_length))

  def add_customer(self, schedule, traveler):
        #update info
        self.update_info(schedule.now)

        #add tarveler object/instance to the created list
        self.queue.append(traveler)

        #check if the server is free
        if self.people_being_served < 1:
            # This customer can be served immediately
            schedule.add_event_after(0, self.start_serving_customer)

  def start_serving_customer(self, schedule):
        self.update_info(schedule.now)

        # Move the customer from the queue to a server
        if len(self.queue) > 0:
            traveler = self.queue.pop(0)
            self.people_being_served += 1

            # Calculate Wait Time: (Current Time - Arrival Time)
            wait_time = schedule.now - traveler.arrival_time
            self.waiting_times.append(wait_time)


            service_time = self.service_distribution.rvs()
            # Schedule when the server will be done
            schedule.add_event_after(
```

```
                service_time,
                self.finish_serving_customer)

  def finish_serving_customer(self, schedule):
        self.update_info(schedule.now)

        # Remove the customer from the server
        self.people_being_served -= 1
        if len(self.queue) > 0 :
            # There are more people in the queue so serve the next
customer
            schedule.add_event_after(0, self.start_serving_customer)


class Airport:
  """
  Manages the arrival of travelers and distributes them to the
shortest queue
  """
  def __init__(self, num_queues, arrival_distribution,
service_distribution):
        #list of queues
        self.queues = []
        for i in range(num_queues):
            self.queues.append(Queues(i, service_distribution))

        #arrival distribution information
        self.arrival_distribution = arrival_distribution
        self.total_travelers = 0


  def add_customer(self, schedule):
        #create traveler
        self.total_travelers += 1
        new_traveler = Traveler(schedule.now, self.total_travelers)

        #need to add customer to the shortest queue using greedy
algorithm
```

```
        shortest_queue = min(self.queues, key=lambda q:len(q.queue) +
q.people_being_served) #added to check for queue + people being
served for the shortest queue
        shortest_queue.add_customer(schedule, new_traveler)

        # Schedule when to add another customer
        schedule.add_event_after(
            self.arrival_distribution.rvs(),
            self.add_customer)

  def open_airport(self, schedule): #used to be run needs to update
        # Schedule when the first customer arrives
        schedule.add_event_after(
            self.arrival_distribution.rvs(),
            self.add_customer)

def run_simulation(arrival_distribution, service_distribution,
num_queues, run_until):
  #need to set up the paramenters here
    schedule = Schedule()

    airport = Airport(num_queues, arrival_distribution,
service_distribution)
    airport.open_airport(schedule)
    while schedule.next_event_time() is not None and
schedule.next_event_time() < run_until:
        schedule.run_next_event()

    # After the last event, we must update each queue
    # up to run_until to close the area-under-curve
    for q in airport.queues:
        q.update_info(run_until)
    #schedule.now = run_until
    return airport, schedule
```

```
#Extended Model
class QueuesExtended(Queues):
    def __init__(self, queue_id, service_distribution, airport):
        # 1. Correctly call the 2-argument init from Cell 2
        super().__init__(queue_id, service_distribution)
        # 2. Add Airport manually
        self.airport = airport

    # Override start to intercept service completion
    def start_serving_customer(self, schedule):
        self.update_info(schedule.now)
        if len(self.queue) > 0:
            traveler = self.queue.pop(0)
            self.people_being_served += 1

            traveler.service_start_time = schedule.now
            wait_time = schedule.now - traveler.arrival_time
            self.waiting_times.append(wait_time)

            service_time = self.service_distribution.rvs()
            # Call EXTENDED finish logic
            schedule.add_event_after(service_time,
self.finish_extended_service, traveler)

    def finish_extended_service(self, schedule, traveler):
        self.update_info(schedule.now)
        # Check Secondary Screening
        if self.airport.check_prob > 0 and np.random.random() <
self.airport.check_prob:
            self.airport.request_senior_officer(schedule, self, traveler)
        else:
            # Clear: Call basic finish (which decrements
people_being_served)
            self.finish_serving_customer(schedule)

    def complete_departure(self, schedule, traveler):
        # Unblock lane
        self.finish_serving_customer(schedule)
```

```
class AirportExtended:
    def __init__(self, num_queues, arrival_dist, service_dist, senior_service_dist,
check_prob):
        self.queues = [QueuesExtended(i, service_dist, self) for i in
range(num_queues)]
        self.arrival_distribution = arrival_dist
        self.senior_service_dist = senior_service_dist
        self.check_prob = check_prob
        self.senior_officer_busy = False
        self.senior_queue = []
        self.total_travelers = 0

    def add_customer(self, schedule):
        self.total_travelers += 1
        new_traveler = Traveler(schedule.now, self.total_travelers)
        shortest_queue = min(self.queues, key=lambda q: len(q.queue) +
q.people_being_served)
        shortest_queue.add_customer(schedule, new_traveler)
        schedule.add_event_after(self.arrival_distribution.rvs(),
self.add_customer)

    def open_airport(self, schedule):
        schedule.add_event_after(self.arrival_distribution.rvs(),
self.add_customer)

    def request_senior_officer(self, schedule, queue_obj, traveler):
        if not self.senior_officer_busy:
            self.senior_officer_busy = True
            self.start_senior_service(schedule, queue_obj, traveler)
        else:
            self.senior_queue.append((queue_obj, traveler))

    def start_senior_service(self, schedule, queue_obj, traveler):
        duration = self.senior_service_dist.rvs()
        schedule.add_event_after(duration, self.finish_senior_service, queue_obj,
traveler)

    def finish_senior_service(self, schedule, queue_obj, traveler):
        queue_obj.complete_departure(schedule, traveler)
        if self.senior_queue:
            next_q, next_t = self.senior_queue.pop(0)
            self.start_senior_service(schedule, next_q, next_t)
        else:
            self.senior_officer_busy = False

def run_extended_simulation(arrival_dist, service_dist, num_queues,
```

```
run_until, senior_dist, check_prob):
    schedule = Schedule()
    airport = AirportExtended(num_queues, arrival_dist, service_dist,
senior_dist, check_prob)
    airport.open_airport(schedule)
    while schedule.next_event_time() is not None and
schedule.next_event_time() < run_until:
        schedule.run_next_event()
    for q in airport.queues:
        q.update_info(run_until)
    return airport
```

Appendix B: The following scripts were used to generate the theoretical verification tables and the optimization plots.

```
# Verification tests

def run_verification_tests():
    #function to help run multiple trials and calculate the avergae
of them
    def run_trials(arrival_dist, service_dist, num_queues, duration):
        results = []
        TRIALS = 1000
        print("Running")

        #loading bar
        for _ in tqdm(range(TRIALS)):
            airport, schedule = run_simulation(arrival_dist,
service_dist, num_queues, duration)       # 2. SNAPSHOT: Collect data
ONLY at the very end
            # Collect data at the end by counting how many people are
standing there right now.
            current_lengths = []
            for q in airport.queues:
                # Just the length of the list at the finish line
```

```
                current_lengths.append(len(q.queue))

            # average queue length across the queues for this
specific instant
            results.append(np.mean(current_lengths))

        # average of the 1000 snapshots
        return np.mean(results)

    #test 1
    print("Test 1: Uniform Service with 6 queues")
    arrival_dist_1 = sts.expon(scale=1/10)
    service_dist_1 = sts.uniform(loc=0, scale=1)
    num_queues_1 = 6
    duration_1 = 100

    result_1 = run_trials(arrival_dist_1, service_dist_1,
num_queues_1, duration_1)

    print(f" Result: {result_1}")

    #checking if it is Mean queue length should be 0.463 ± 0.025 (per
queue)
    if 0.438 <= result_1 <= 0.488:
        print("Passed")
    else:
        print("Failed")

    #test 2
    print("Test 2: Truncated Normal with 14 Queues")
    arrival_dist_2 = sts.expon(scale=1/10)
    service_dist_2 = sts.truncnorm(-1, np.inf, loc=1, scale=1)
    num_queues_2 = 14
    duration_2 = 200
    result_2 = run_trials(arrival_dist_2, service_dist_2,
num_queues_2, duration_2)
    print(f" Result: {result_2}")
```

```
    #checking if it is Mean queue length should be 0.566 ± 0.025 (per
queue)
    if 0.541 <= result_2 <= 0.591:
        print("Passed")
    else:
        print("Failed")

run_verification_tests()
```

```
#verification of basic model (theoretical Comparison)

def calculate_theoretical_Lq(k, lam, mu, sigma):
    # We treat verification as single M/G/1 with scaled arrival
    lam_per_queue = lam / k
    rho = lam_per_queue * mu
    if rho >= 1: return float('inf')
    var = sigma**2
    return ((rho**2 + (lam_per_queue)**2 * var) / (2 * (1 - rho)))

def verify_basic_model():
    print("VERIFICATION: Theory vs Simulation (M/G/1 Equivalent)")
    print("running long simulations to minimize warm-up bias")

    lam = 10

    # uniform distribution
    k1 = 6
    mu1 = 0.5
    sigma1 = np.sqrt(1/12)

    theory_1 = calculate_theoretical_Lq(k1, lam, mu1, sigma1)

    # Simulation: 1 queue, scaled arrival
    lam_1_scaled = lam / k1
    sim_res_1 = []
```

```
    # Running 30 trials for stability
    for _ in range(30):
        # num_queues=1 to match M/G/1 math
        airport, _ = run_simulation(sts.expon(scale=1/lam_1_scaled),
sts.uniform(loc=0, scale=1), 1, 20000)
        sim_res_1.append(np.mean([q.area_under_queue_curve/20000 for
q in airport.queues]))

    sim_val_1 = np.mean(sim_res_1)
    print(f"CASE 1 (Uniform): Theory {theory_1:.3f} | Sim
{sim_val_1:.3f} -> {'PASS' if abs(theory_1 - sim_val_1) < 0.15 else
'FAIL'}")

    #case 2: truncated normal
    k2 = 14
    mu2 = 1.0
    sigma2 = 1.0

    a, b = (0 - mu2)/sigma2, np.inf
    m_s, v_s = sts.truncnorm.stats(a, b, loc=mu2, scale=sigma2,
moments='mv')
    theory_2 = calculate_theoretical_Lq(k2, lam, m_s, np.sqrt(v_s))

    lam_2_scaled = lam / k2
    sim_res_2 = []

    for _ in range(30):
        airport, _ = run_simulation(sts.expon(scale=1/lam_2_scaled),
sts.truncnorm(a, b, loc=mu2, scale=sigma2), 1, 20000)
        sim_res_2.append(np.mean([q.area_under_queue_curve/20000 for
q in airport.queues]))

    sim_val_2 = np.mean(sim_res_2)

    # Increased tolerance slightly because TruncNorm is stochastic
    print(f"CASE 2 (Normal):  Theory {theory_2:.3f} | Sim
{sim_val_2:.3f} -> {'PASS' if abs(theory_2 - sim_val_2) < 0.3 else
'FAIL'}")
```

```
    print("-" * 60)

verify_basic_model()
```

Appendix C Plots

```
def plot_distribution_check():
    print("Checking Random Distributions")
    # Check Arrivals (Exponential)
    data_arrival = sts.expon(scale=1/10).rvs(size=10000)
    x = np.linspace(0, 1, 100)
    pdf = sts.expon(scale=1/10).pdf(x)

    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.hist(data_arrival, bins=50, density=True, alpha=0.6,
color='blue', label='Sim Data')
    plt.plot(x, pdf, 'r-', lw=2, label='Theory PDF')
    plt.title('Arrivals: Exponential(Rate=10)')
    plt.legend()

    # Check Service (Truncated Normal)
    mu, sigma = 0.5, 1/6
    a, b = (0 - mu) / sigma, np.inf
    data_service = sts.truncnorm(a, b, loc=mu,
scale=sigma).rvs(size=10000)
    x2 = np.linspace(0, 1.5, 100)
    pdf2 = sts.truncnorm(a, b, loc=mu, scale=sigma).pdf(x2)

    plt.subplot(1, 2, 2)
    plt.hist(data_service, bins=50, density=True, alpha=0.6,
color='green', label='Sim Data')
    plt.plot(x2, pdf2, 'r-', lw=2, label='Theory PDF')
    plt.title('Service: TruncNorm(Mean=0.5)')
    plt.legend()
    plt.show()
```

```
plot_distribution_check()
```